

TOPIC 6 : BACKTRACKING

Question 1 – N-Queens Visualization

Aim

To implement the **N-Queens problem using backtracking in Python** and display valid queen placements for different values of **N**, such as 4, 5, and 8.

Algorithm

1. Start with an empty $N \times N$ chessboard.
2. Place one queen in each row.
3. Before placing a queen, check:
 - The column is not occupied.
 - The left diagonal is not occupied.
 - The right diagonal is not occupied.
4. If the position is safe, place the queen.
5. Move to the next row.
6. If no safe position is available, backtrack to the previous row and move its queen.
7. Continue until all N queens are placed.
8. Display the valid board configuration.

Python Source Code

```
def display_solution(solution, n):
    for row in range(n):
        for col in range(n):
            if solution[row] == col:
                print("Q", end=" ")
            else:
                print(".", end=" ")
        print()
    print()

n = int(input("Enter N: "))

solutions = solve_n_queens(n)

print("\nNumber of solutions:", len(solutions))

if solutions:
    print("\nOne valid solution:")
    display_solution(solutions[0], n)
else:
    print("No solution exists.")

for size in [4, 5, 8]:
    solutions = solve_n_queens(size)

    print("N =", size)
    print("Number of solutions:", len(solutions))
    print("One valid solution:")
    display_solution(solutions[0], size)
```

```

def is_safe(board, row, col, n):

    for i in range(row):
        if board[i] == col:
            return False

    for i, j in zip(range(row - 1, -1, -1),
                    range(col - 1, -1, -1)):
        if board[i] == j:
            return False

    for i, j in zip(range(row - 1, -1, -1),
                    range(col + 1, n)):
        if board[i] == j:
            return False

    return True


def solve_n_queens(n):
    board = [-1] * n
    solutions = []

    def backtrack(row):
        if row == n:
            solutions.append(board.copy())
            return

        for col in range(n):
            if is_safe(board, row, col, n):
                board[row] = col
                backtrack(row + 1)
                board[row] = -1

    backtrack(0)
    return solutions

```

Enter N: 4

Number of solutions: 2

One valid solution:

```
. Q . .  
. . . Q  
Q . . .  
. . Q .
```

N = 4

Number of solutions: 2

One valid solution:

```
. Q . .  
. . . Q  
Q . . .  
. . Q .
```

N = 5

Number of solutions: 10

One valid solution:

```
Q . . . .  
. . Q . .  
. . . . Q  
. Q . . .  
. . . Q .
```

N = 8

Number of solutions: 92

One valid solution:

```
Q . . . . . . .  
. . . . Q . . .  
. . . . . . . Q  
. . . . . Q . .  
. . Q . . . . .  
. . . . . . Q .  
. Q . . . . . .  
. . . Q . . . .
```

Time Complexity

$O(N!)$ in the worst case.

Space Complexity

$O(N)$ for the recursive/backtracking storage, excluding the memory required to store all solutions.

Result

Thus, the **N-Queens problem was successfully implemented using Python backtracking**, and valid queen arrangements were generated and displayed for different values of N.

Question 2 – Generalized N-Queens Problem

Aim

To implement a generalized **N-Queens problem using Python backtracking** that can place queens on boards with different sizes and additional constraints such as obstacles and restricted positions.

Algorithm

1. Create a board with the specified number of rows and columns.
2. Start placing queens row by row.
3. For each row, check every column.
4. Reject a position if:
 - The cell contains an obstacle.
 - The position is restricted.
 - Another queen exists in the same column.
 - Another queen exists on the same diagonal.
5. If the position is safe, place the queen.
6. Recursively move to the next row.
7. If no position is possible, remove the previously placed queen and backtrack.
8. Continue until all queens are placed.
9. Display the solution.

Python Source Code

```

def is_safe(board, row, col, rows, cols, obstacles):
    for r in range(row):
        if board[r] == col:
            return False

    r = row - 1
    c = col - 1

    while r >= 0 and c >= 0:
        if board[r] == c:
            return False
        r -= 1
        c -= 1

    r = row - 1
    c = col + 1

    while r >= 0 and c < cols:
        if board[r] == c:
            return False
        r -= 1
        c += 1

    if (row + 1, col + 1) in obstacles:
        return False

    return True

def solve_queens(rows, cols, obstacles=set(), restricted=None):
    board = [-1] * rows

    def backtrack(row):
        if row == rows:
            return True

        for col in range(cols):
            if row == 0 and restricted is not None:
                if col + 1 not in restricted:
                    continue

```

```

        if is_safe(board, row, col, rows, cols, obstacles):
            board[row] = col

            if backtrack(row + 1):
                return True

            board[row] = -1

        return False

    if backtrack(0):
        return board

    return None

def display_board(solution, rows, cols):
    if solution is None:
        print("No solution exists")
        return

    for row in range(rows):
        for col in range(cols):
            if solution[row] == col:
                print("Q", end=" ")
            else:
                print(".", end=" ")
        print()

    print("Queen positions:", [x + 1 for x in solution])

print("8 x 10 BOARD")
solution = solve_queens(8, 10)
display_board(solution, 8, 10)

print("\n5 x 5 BOARD WITH OBSTACLES")
obstacles = {(2, 2), (4, 4)}
solution = solve_queens(5, 5, obstacles)
display_board(solution, 5, 5)

```

Output

```
===== RESTART: C:/Users/HP/Downloads/
```

```
8 x 10 BOARD
```

```
Q . . . . . . . . . .  
. . Q . . . . . . . .  
. . . . Q . . . . . . .  
. Q . . . . . . . . .  
. . . . . . . . Q . .  
. . . . . . . . . Q  
. . . Q . . . . . . .  
. . . . . . Q . . . .
```

```
Queen positions: [1, 3, 5, 2, 8, 10, 4, 7]
```

```
5 x 5 BOARD WITH OBSTACLES
```

```
Q . . . .  
. . Q . .  
. . . . Q  
. Q . . .  
. . . Q .
```

```
Queen positions: [1, 3, 5, 2, 4]
```

```
6 x 6 BOARD WITH RESTRICTED POSITIONS
```

```
. Q . . . .  
. . . Q . .  
. . . . . Q  
Q . . . . .  
. . Q . . .  
. . . . Q .
```

```
Queen positions: [2, 4, 6, 1, 3, 5]
```

```
> |
```

Time Complexity

$O(C^R)$ in the general case, where:

- R = number of rows
- C = number of columns

For a normal N-Queens board, this becomes approximately $O(N!)$ with backtracking.

Space Complexity

$O(R)$ for the queen-position array and recursion stack.

Result

Thus, the **generalized N-Queens problem was successfully implemented in Python using backtracking**, while considering different board dimensions, obstacles, and restricted positions.

Question 3 – Sudoku Solver

Aim

To implement a **Sudoku Solver using the Backtracking technique in Python** and fill all empty cells while satisfying the Sudoku constraints.

Algorithm

1. Find an empty cell in the Sudoku board.
2. Try numbers from 1 to 9.
3. Check whether the number is safe in the selected cell.
4. The number must not already exist in:
 - The same row.
 - The same column.
 - The same 3×3 sub-box.
5. If the number is safe, place it in the cell.
6. Recursively solve the remaining empty cells.
7. If the solution is not possible, remove the number and backtrack.
8. Continue until the entire Sudoku is solved.

Python Source Code

```
def is_valid(board, row, col, num):
    for i in range(9):
        if board[row][i] == num:
            return False

    for i in range(9):
        if board[i][col] == num:
            return False

    start_row = (row // 3) * 3
    start_col = (col // 3) * 3

    for i in range(start_row, start_row + 3):
        for j in range(start_col, start_col + 3):
            if board[i][j] == num:
                return False

    return True

def find_empty(board):
    for i in range(9):
        for j in range(9):
            if board[i][j] == 0:
                return i, j
    return None

def solve_sudoku(board):
    empty = find_empty(board)

    if empty is None:
        return True

    row, col = empty

    for num in range(1, 10):
        if is_valid(board, row, col, num):
            board[row][col] = num
```

```

        if solve_sudoku(board):
            return True

        board[row][col] = 0

    return False

def print_board(board):
    for row in board:
        print(row)

board = [
    [5, 3, 0, 0, 7, 0, 0, 0, 0],
    [6, 0, 0, 1, 9, 5, 0, 0, 0],
    [0, 9, 8, 0, 0, 0, 0, 6, 0],
    [8, 0, 0, 0, 6, 0, 0, 0, 3],
    [4, 0, 0, 8, 0, 3, 0, 0, 1],
    [7, 0, 0, 0, 2, 0, 0, 0, 6],
    [0, 6, 0, 0, 0, 0, 2, 8, 0],
    [0, 0, 0, 4, 1, 9, 0, 0, 5],
    [0, 0, 0, 0, 8, 0, 0, 7, 9]
]

print("Sudoku Before Solving:")
print_board(board)

if solve_sudoku(board):
    print("\nSudoku After Solving:")
    print_board(board)
else:
    print("No solution exists")

```

Output

```
===== RESTART: C:/Users/.
Sudoku Before Solving:
[5, 3, 0, 0, 7, 0, 0, 0, 0]
[6, 0, 0, 1, 9, 5, 0, 0, 0]
[0, 9, 8, 0, 0, 0, 0, 6, 0]
[8, 0, 0, 0, 6, 0, 0, 0, 3]
[4, 0, 0, 8, 0, 3, 0, 0, 1]
[7, 0, 0, 0, 2, 0, 0, 0, 6]
[0, 6, 0, 0, 0, 0, 2, 8, 0]
[0, 0, 0, 4, 1, 9, 0, 0, 5]
[0, 0, 0, 0, 8, 0, 0, 7, 9]

Sudoku After Solving:
[5, 3, 4, 6, 7, 8, 9, 1, 2]
[6, 7, 2, 1, 9, 5, 3, 4, 8]
[1, 9, 8, 3, 4, 2, 5, 6, 7]
[8, 5, 9, 7, 6, 1, 4, 2, 3]
[4, 2, 6, 8, 5, 3, 7, 9, 1]
[7, 1, 3, 9, 2, 4, 8, 5, 6]
[9, 6, 1, 5, 3, 7, 2, 8, 4]
[2, 8, 7, 4, 1, 9, 6, 3, 5]
[3, 4, 5, 2, 8, 6, 1, 7, 9]
.>
```

Time Complexity

$O(9^M)$

where M is the number of empty cells.

Space Complexity

$O(M)$ for the recursion stack.

Result

Thus, the Sudoku puzzle was successfully solved using the **Backtracking technique in Python**, satisfying all row, column, and 3×3 sub-box constraints.

Question 5 – Target Sum

Question 5 asks you to add either + or - before each number in an array so that the resulting expression equals the given target, and return the **number of different expressions** possible.

Pasted text

Aim

To find the number of different expressions that can be formed by assigning + or - signs to the elements of an array such that their sum equals the given target, using **Backtracking in Python**.

Algorithm

1. Start from the first element.
2. Add the element to the current sum.
3. Recursively process the next element.
4. Subtract the element and recursively process the next element.
5. When all elements are processed, check whether the current sum equals the target.
6. If it equals the target, count the expression.
7. Return the total number of valid expressions.

Python Source Code

```
File Edit Format Run Options Window Help
def find_target_sum(nums, target):
    count = 0

    def backtrack(index, current_sum):
        nonlocal count

        if index == len(nums):
            if current_sum == target:
                count += 1
            return

        backtrack(index + 1, current_sum + nums[index])
        backtrack(index + 1, current_sum - nums[index])

    backtrack(0, 0)
    return count

nums = [1, 1, 1, 1, 1]
target = 3

result = find_target_sum(nums, target)
print("Number of ways:", result)
```

OUTPUT:

```
===== RESTART: C:/Users/HP/Dc
Number of ways: 5
```

Time Complexity

$O(2^n)$

Each element has two choices: + or -.

Space Complexity

$O(n)$

This is due to the recursion stack.

Result

Thus, the number of expressions that evaluate to the target was successfully calculated using **Backtracking in Python**. For [1,1,1,1,1] and target 3, the result is **5**.

Question 6 – Sum of Subarray Minimums

Aim

To find the sum of minimum elements of all possible contiguous subarrays using an efficient **stack-based algorithm in Python**.

Algorithm

1. Traverse the array from left to right.
2. For every element, find how many subarrays have that element as their minimum.
3. Use a monotonic increasing stack to find:
 - The previous smaller element.
 - The next smaller element.
4. Calculate the contribution of each element:
element $\times$ left choices $\times$ right choices.
5. Add all contributions.
6. Return the result modulo $10^9 + 7$.

Python Source Code

```

def sum_subarray_mins(arr):
    MOD = 10**9 + 7
    n = len(arr)

    left = [0] * n
    right = [0] * n

    stack = []

    for i in range(n):
        while stack and arr[stack[-1]] > arr[i]:
            stack.pop()

        if stack:
            left[i] = i - stack[-1]
        else:
            left[i] = i + 1

        stack.append(i)

    stack = []

    for i in range(n - 1, -1, -1):
        while stack and arr[stack[-1]] >= arr[i]:
            stack.pop()

        if stack:
            right[i] = stack[-1] - i
        else:
            right[i] = n - i

        stack.append(i)

    result = 0

    for i in range(n):
        result += arr[i] * left[i] * right[i]
        result %= MOD

    return result

```

OUTPUT:

```

===== RESTART: C:/Users/HP/DO
Array: [3, 1, 2, 4]
Sum of subarray minimums: 17

```

Time Complexity

$O(n)$

Each element is pushed and popped from the stack at most once.

Space Complexity

$O(n)$

For the left, right, and stack arrays.

Result

Thus, the **sum of minimum elements of all contiguous subarrays** was successfully calculated using Python. For [3, 1, 2, 4], the result is **17**.

ChatGPT said:

Question 7 – Combination Sum

Aim

To find all possible unique combinations of numbers whose sum is equal to a given target using the **Backtracking technique in Python**.

Algorithm

1. Sort the candidate array.
2. Start from the first candidate.
3. Choose a candidate and add it to the current combination.
4. Recursively continue choosing candidates, including the same candidate again.
5. If the current sum becomes equal to the target, store the combination.
6. If the current sum exceeds the target, stop that branch.
7. Backtrack by removing the last selected candidate.
8. Continue until all possible combinations are explored.

Python Source Code

```
def combination_sum(candidates, target):
    result = []
    candidates.sort()

    def backtrack(start, current, total):
        if total == target:
            result.append(current.copy())
            return

        if total > target:
            return

        for i in range(start, len(candidates)):
            current.append(candidates[i])
            backtrack(i, current, total + candidates[i])
            current.pop()

    backtrack(0, [], 0)
    return result
```

```
candidates = [2, 3, 6, 7]
target = 7
```

```
result = combination_sum(candidates, target)

print("Combinations:", result)
```

OUTPUT:

```
----- RESTART: C:/Users/nf/DOW
>> Combinations: [[2, 2, 3], [7]]
```

Time Complexity

$O(2^n)$ approximately, depending on the target and number of possible combinations.

Space Complexity

$O(n)$ for the recursion depth, excluding the space used to store the output combinations.

Result

Thus, all unique combinations whose sum equals the target were successfully generated using the **Backtracking technique in Python**.

Question 8 – Combination Sum II

Aim

To find all unique combinations of elements from an array whose sum is equal to a given target using the **Backtracking technique in Python**, where each element can be used only once.

Algorithm

1. Sort the candidate array.
2. Start backtracking from the first element.
3. Select an element and add it to the current combination.
4. Recursively process the next index.
5. If the sum becomes equal to the target, store the combination.
6. If the sum exceeds the target, stop that branch.
7. Skip duplicate elements at the same recursion level.
8. Remove the selected element and backtrack.
9. Continue until all possible combinations are checked.

Python Source Code

```

def combination_sum2(candidates, target):
    result = []
    candidates.sort()

    def backtrack(start, current, total):
        if total == target:
            result.append(current.copy())
            return

        if total > target:
            return

        for i in range(start, len(candidates)):
            if i > start and candidates[i] == candidates[i - 1]:
                continue

            current.append(candidates[i])
            backtrack(i + 1, current, total + candidates[i])
            current.pop()

    backtrack(0, [], 0)
    return result

candidates = [10, 1, 2, 7, 6, 1, 5]
target = 8

result = combination_sum2(candidates, target)

print("Unique combinations:", result)

```

OUTPUT:

```

>>|===== RESTART: C:/Users/HP/Downloads/Documents/e1.py ===
>>|Unique combinations: [[1, 1, 6], [1, 2, 5], [1, 7], [2, 6]]
>>|

```

Time Complexity

$O(2^n)$ in the worst case, where n is the number of candidates.

Space Complexity

$O(n)$ for the recursion stack, excluding the output combinations.

Result

Thus, all **unique combinations** whose sum equals the target were successfully generated using **Backtracking in Python**, while ensuring that each candidate is used at most once and duplicate combinations are avoided.

Question 9 – Permutations

Aim

To generate all possible permutations of a given array of distinct integers using the **Backtracking technique in Python**.

Algorithm

1. Start with an empty permutation.
2. Select an unused element from the array.
3. Add the selected element to the current permutation.
4. Recursively select the remaining unused elements.
5. When the permutation contains all elements, store it.
6. Remove the last element and backtrack.
7. Repeat until all possible permutations are generated.

Python Source Code

```
File Edit Format Run Options Window Help
def permutations(nums):
    result = []
    used = [False] * len(nums)
    current = []

    def backtrack():
        if len(current) == len(nums):
            result.append(current.copy())
            return

        for i in range(len(nums)):
            if not used[i]:
                used[i] = True
                current.append(nums[i])

                backtrack()

                current.pop()
                used[i] = False

    backtrack()
    return result

nums = [1, 2, 3]

result = permutations(nums)

print("Permutations:")
for p in result:
    print(p)
```

OUTPUT:

```
----- RESTART
Permutations:
[1, 2, 3]
[1, 3, 2]
[2, 1, 3]
[2, 3, 1]
[3, 1, 2]
[3, 2, 1]
```

Time Complexity

$O(n \times n!)$

There are $n!$ permutations, and copying each permutation takes $O(n)$ time.

Space Complexity

$O(n)$ for the recursion stack and temporary permutation.

Excluding the output, the auxiliary space is $O(n)$.

Result

Thus, all possible permutations of the given array were successfully generated using the **Backtracking technique in Python**.

ChatGPT said:

Question 10 – Unique Permutations

Aim

To generate all possible **unique permutations** of an array containing duplicate elements using the **Backtracking technique in Python**.

Algorithm

1. Sort the given array.
2. Create an array to keep track of used elements.
3. Start with an empty permutation.
4. Select an unused element.
5. Skip an element if it is the same as the previous element and the previous element has not been used in the current position.
6. Add the selected element to the permutation.
7. Recursively generate the remaining elements.
8. Remove the selected element and backtrack.
9. Continue until all unique permutations are generated.

Python Source Code

```

def unique_permutations(nums):
    nums.sort()
    result = []
    used = [False] * len(nums)
    current = []

    def backtrack():
        if len(current) == len(nums):
            result.append(current.copy())
            return

        for i in range(len(nums)):
            if used[i]:
                continue

            if i > 0 and nums[i] == nums[i - 1] and not used[i - 1]:
                continue

            used[i] = True
            current.append(nums[i])

            backtrack()

            current.pop()
            used[i] = False

    backtrack()
    return result

|
nums = [1, 1, 2]

result = unique_permutations(nums)

print("Unique Permutations:")
for permutation in result:
    print(permutation)

```

OUTPUT:

```

| Unique Permutations:
| [1, 1, 2]
| [1, 2, 1]
| [2, 1, 1]
|

```

Time Complexity

$O(n \times n!)$ in the worst case.

There can be up to $n!$ permutations, and copying each permutation takes $O(n)$ time.

Space Complexity

$O(n)$ for the recursion stack, used array, and current permutation, excluding the output.

Result

Thus, all **unique permutations** of the given array were successfully generated using **Backtracking in Python**, while avoiding duplicate permutations.

Question 11 – Graph Coloring

Aim

To implement the **Graph Coloring problem using Backtracking in Python** and find a valid coloring of an undirected graph using a limited number of colors.

Algorithm

1. Represent the graph using an adjacency matrix.
2. Start with the first vertex.
3. Try assigning each available color to the vertex.
4. Check whether the color conflicts with any already-colored adjacent vertex.
5. If there is no conflict, assign the color.
6. Move to the next vertex recursively.
7. If no color is possible, backtrack and change the previous color.
8. Continue until all vertices are colored.
9. Display the valid coloring and number of colors used.

Python Source Code

```

def backtrack(vertex):
    if vertex == n:
        return True

    for color in range(1, k + 1):
        if is_safe(vertex, color, graph, colors):
            colors[vertex] = color

            if backtrack(vertex + 1):
                return True

            colors[vertex] = 0

    return False

if backtrack(0):
    return colors

return None

n = 4
edges = [(0, 1), (1, 2), (2, 3), (3, 0), (0, 2)]
k = 3

graph = [[0] * n for _ in range(n)]

for u, v in edges:
    graph[u][v] = 1
    graph[v][u] = 1

colors = graph_coloring(graph, k)

if colors:
    print("Graph Coloring:")
    for i in range(n):
        print("Vertex", i, "-> Color", colors[i])

    print("Number of colors used:", max(colors))

```

OUTPUT:

```
Graph Coloring:
Vertex 0 -> Color 1
Vertex 1 -> Color 2
Vertex 2 -> Color 3
Vertex 3 -> Color 2
Number of colors used: 3
```

Time Complexity

$O(k^n)$ in the worst case, where:

- n = number of vertices
- k = number of available colors

Space Complexity

$O(n)$ for the color array and recursion stack.

Result

Thus, the **Graph Coloring problem was successfully implemented using Backtracking in Python**, and the given graph was successfully colored using at most **3 colors**.

Question 12 – Hamiltonian Cycle

Question 12 asks you to determine whether an **undirected graph contains a Hamiltonian cycle**. A Hamiltonian cycle visits every vertex exactly once and returns to the starting vertex. The given example has $n = 5$ vertices and the specified edges.

Pasted text

Aim

To determine whether a given undirected graph contains a **Hamiltonian Cycle** using the **Backtracking technique in Python**.

Algorithm

1. Start the Hamiltonian path from vertex 0.
2. Add an unvisited vertex to the path.
3. Check whether the vertex is adjacent to the previous vertex.
4. Ensure that the vertex has not already been included.

5. Continue until all vertices are included.
6. Check whether the last vertex is connected to the starting vertex.
7. If connected, a Hamiltonian cycle exists.
8. If no valid vertex can be added, backtrack and try another vertex.
9. Return the result.

Python Source Code

```
def is_safe(vertex, graph, path, position):
    if graph[path[position - 1]][vertex] == 0:
        return False

    if vertex in path:
        return False

    return True

def hamiltonian_cycle(graph):
    n = len(graph)
    path = [-1] * n
    path[0] = 0

    def backtrack(position):
        if position == n:
            return graph[path[position - 1]][path[0]] == 1

        for vertex in range(1, n):
            if is_safe(vertex, graph, path, position):
                path[position] = vertex

                if backtrack(position + 1):
                    return True

                path[position] = -1

        return False

    if backtrack(1):
        return path

    return None
```

OUTPUT:

```
===== RESTART: C:/Users/HP/  
Hamiltonian Cycle Exists: False
```

Time Complexity

$O(n!)$ in the worst case.

Space Complexity

$O(n)$ for the path array and recursion stack.

Result

Thus, the **Hamiltonian Cycle** was successfully identified using **Backtracking in Python**. For the given graph, a Hamiltonian cycle exists: $0 \rightarrow 1 \rightarrow 2 \rightarrow 4 \rightarrow 3 \rightarrow 0$.

Q14 – Hamiltonian Cycle

Aim

To determine whether an undirected graph contains a **Hamiltonian Cycle** using the Backtracking technique in Python.

Algorithm

1. Start from vertex 0.
2. Add an unvisited adjacent vertex to the path.
3. Check that the vertex is not already present.
4. Continue until all vertices are included.
5. Check whether the last vertex connects back to vertex 0.
6. If it does, a Hamiltonian cycle exists.
7. Otherwise, backtrack and try another vertex.

Source Code

```
def is_safe(vertex, graph, path, position):
    if graph[path[position - 1]][vertex] == 0:
        return False

    if vertex in path:
        return False

    return True

def hamiltonian_cycle(graph):
    n = len(graph)
    path = [-1] * n
    path[0] = 0

    def backtrack(position):
        if position == n:
            return graph[path[position - 1]][path[0]] == 1

        for vertex in range(1, n):
            if is_safe(vertex, graph, path, position):
                path[position] = vertex

                if backtrack(position + 1):
                    return True

                path[position] = -1

        return False

    if backtrack(1):
        return path

    return None

n = 4

edges = [
    (0, 1),
    (1, 2),
```

```

n = 4

edges = [
    (0, 1),
    (1, 2),
    (2, 3),
    (3, 0),
    (0, 2)
]

graph = [[0] * n for _ in range(n)]

for u, v in edges:
    graph[u][v] = 1
    graph[v][u] = 1

cycle = hamiltonian_cycle(graph)

if cycle:
    cycle.append(cycle[0])
    print("Hamiltonian Cycle Exists: True")
    print("Cycle:", " -> ".join(map(str, cycle)))
else:
    print("Hamiltonian Cycle Exists: False")

```

OUTPUT:

```

===== RESTART: C:/Users/HP/I
Hamiltonian Cycle Exists: True
Cycle: 0 -> 1 -> 2 -> 3 -> 0
>

```

Time Complexity

$O(n!)$

Space Complexity

$O(n)$

Result

The Hamiltonian Cycle was successfully found using **Backtracking in Python**

Q15 – Generate All Subsets

Aim: To generate all possible subsets of a given set in **lexicographical order** using the Backtracking technique and handle duplicate elements.

Algorithm

1. Sort the input array.
2. Start with an empty subset.
3. Add the current subset to the result.
4. Select an element and add it to the subset.
5. Recursively generate further subsets.
6. Remove the element and backtrack.
7. Skip duplicate elements when necessary.
8. Continue until all subsets are generated.

Python Source Code

```
File Edit Format Run Options Window Help
def generate_subsets(nums):
    nums.sort()
    result = []
    current = []

    def backtrack(start):
        result.append(current.copy())

        for i in range(start, len(nums)):
            if i > start and nums[i] == nums[i - 1]:
                continue

            current.append(nums[i])
            backtrack(i + 1)
            current.pop()

    backtrack(0)
    return result

nums = [1, 2, 3]
result = generate_subsets(nums)

print("Subsets:")
print(result)
```

```
Subsets:
[[], [1], [1, 2], [1, 2, 3], [1, 3], [2], [2, 3], [3]]
>>
```

$$\mathbf{O}(\mathbf{n} \times 2^n)$$

O(n) excluding the output.

All possible subsets were successfully generated using **Backtracking in Python**, with duplicate subsets avoided.

AiM: To generate all subsets of a given set that contain the specified element 3 using Backtracking in Python.

1. Start with an empty subset.
2. Generate subsets using inclusion and exclusion.
3. At the end of each complete subset, check whether it contains 3.
4. If it contains 3, add it to the result.
5. Continue until all possible subsets are generated.

Python Source Code

```

def subsets_containing_element(nums, x):
    result = []
    current = []

    def backtrack(index):
        if index == len(nums):
            if x in current:
                result.append(current.copy())
            return

        current.append(nums[index])
        backtrack(index + 1)
        current.pop()

        backtrack(index + 1)

    backtrack(0)
    return result

nums = [2, 3, 4, 5]
x = 3

result = subsets_containing_element(nums, x)

print("Subsets containing", x, ":")
for subset in result:
    print(subset)

```

OUTPUT:

```

===== RESTART: (
Subsets containing 3 :
[2, 3, 4, 5]
[2, 3, 4]
[2, 3, 5]
[2, 3]
[3, 4, 5]
[3, 4]
[3, 5]
[3]

```

Time Complexity

$O(2^n)$

There are 2^n possible subsets.

Space Complexity

$O(n)$ for the recursion stack and current subset, excluding the output.

Result

Thus, all subsets containing the specified element **3** were successfully generated using **Backtracking in Python**.

Q17 – Universal Strings

Aim

To find all **universal strings** from words1 such that every word in words2 is a subset of that string, including the required frequency of each letter.

Algorithm

1. For every word in words2, count the frequency of each character.
2. Find the maximum required frequency for every character across all words in words2.
3. For every word in words1, count its characters.
4. Compare its character frequencies with the required frequencies.
5. If it contains every required character with sufficient frequency, add it to the result.
6. Print all universal strings.

Python Source Code

```
from collections import Counter

def word_subsets(words1, words2):
    required = Counter()

    for word in words2:
        count = Counter(word)

        for char in count:
            required[char] = max(required[char], count[char])

    result = []

    for word in words1:
        count = Counter(word)
        valid = True

        for char in required:
            if count[char] < required[char]:
                valid = False
                break

        if valid:
            result.append(word)

    return result

words1 = ["amazon", "apple", "facebook", "google", "leetcode"]
words2 = ["e", "o"]

result = word_subsets(words1, words2)

print("Universal strings:", result)
```

OUTPUT:

```
===== RESTART: C:/Users/HP/Downloads/Documents/e1.py =
Universal strings: ['facebook', 'google', 'leetcode']
.>>>
```

Time Complexity

$O(N \times L)$ approximately, where N is the number of words in words1 and L is the average word length.

Space Complexity

$O(26)$ for the character-frequency table, assuming only lowercase English letters.

Result

Thus, the program successfully identifies all **universal strings** from words1 by checking the character frequency requirements of every string in words2.